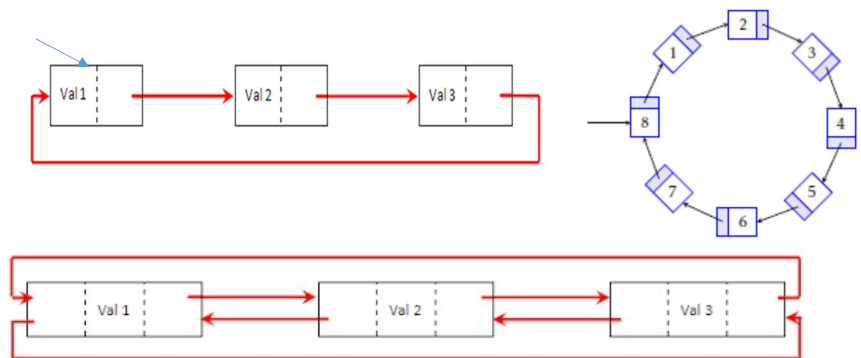


Table des matières

Liste circulaire dynamique simple	2
Avantages des listes circulaires chaînées :	3
Structure de données d'une liste circulaire	4
Algorithmes	5
Initialisation	5
estVideLC : tester si une liste est vide	6
estPleinLC : Tester si une liste est vide	6
Parcourir la liste	7
Version 1 du parcours itératif	8
Version 2 du parcours itératif	9
Version 3 du parcours itératif	9
Version Récursive	10
Algorithmes d'insertions	11
Insertion d'un élément dans une liste vide	11
Insertion d'un élément dans une liste circulaire vide	12
Insertion avant d'un élément dans une liste chaînée circulaire non vide	13
Principe	13
Adresse d'insertion	15
Insertion après d'un élément dans une liste chaînée circulaire non vide	16
Principe	16
Choix d'implémentation	18
Insertion en tête	19
Insertion d'un élément en tête dans une liste circulaire non vide	20
Insertion en queue	21
Insertion d'un élément en queue dans une liste circulaire non vide	22
Algorithmes de suppressions	23
Suppression d'une cellule qui suit la cellule courante	23
Suppression de la liste	25
Principe	25
Version récursive	27

LISTE CIRCULAIRE DYNAMIQUE



Une liste circulaire est une liste où toutes les cellules sont connectées pour former un cercle.

- ✓ Il n'y a pas de NULL à la fin (càd la valeur NULL n'est plus un indicateur de fin).
- ✓ Une liste chaînée circulaire est une liste dans laquelle la queue pointe sur la tête.
- ✓ L'adresse de la liste peut être l'adresse de n'importe quelle cellule de la liste.
- ✓ Une liste circulaire peut être une liste liée circulaire simple ou une liste liée doublement circulaire.

Avantages des listes circulaires chaînées :

- 1) N'importe quelle cellule peut être un point de départ (càd adresse de liste).
- 2) On peut parcourir toute la liste en partant de n'importe quelle cellule. Nous avons juste besoin de nous arrêter lorsque la première cellule visitée est à nouveau visitée.
- 3) Utile pour la mise en **œuvre rapide de la file d'attente**. Contrairement aux implémentations précédentes (une seule adresse ou deux adresses), nous n'avons pas besoin de maintenir deux pointeurs pour l'avant et l'arrière. Si nous utilisons une liste chaînée circulaire, nous pouvons maintenir un pointeur vers la dernière cellule insérée et le front peut toujours être obtenu comme avant-dernier.
- 4) Les listes circulaires sont utiles dans les applications pour faire le tour de la liste à plusieurs reprises. Par exemple, lorsque plusieurs applications s'exécutent sur un PC, il est courant que le **système d'exploitation** place les applications en cours d'exécution sur une liste, puis les parcourt, en donnant à chacune d'elles une tranche de temps pour s'exécuter, puis en les faisant attendre. Tandis que le CPU est donné à une autre application. Il est pratique pour le système d'exploitation d'utiliser une liste circulaire afin que lorsqu'il atteint la fin de la liste, il puisse passer au début de la liste.

Structure de données d'une liste circulaire

Même que celle d'une liste non circulaire.

```
typedef struct cellule *ListeC ;  
struct cellule{  
    TElement donnee ;  
    ListeC suivant ;  
};
```

Liste



ListeC

Algorithmes

Initialisation

Fonction initLC : [] → Liste

Comme pour une liste non circulaire, on initialise la liste à NULL :

Fonction initLC() → Liste

Début

 Renvoyer NULL

FIN

estVideLC : tester si une liste est vide

Fonction estVideLC : Liste → Booléenne

Fonction estVideLC(l) → Booléenne

P.F l : Liste (E)

Debut

 Renvoyer l=null

Fin

estPleinLC : Tester si une liste est vide

Fonction estPleinLC : ListeC → Booléenne

Fonction estPleinLC(l) → Booléenne

P.F l : Liste (E)

Debut

 Renvoyer faux

Fin

Parcourir la liste

Hypothèse : on suppose que la liste est non vide.

Principe :

On peut parcourir toute la liste en partant de n'importe quelle cellule.

Nous avons juste besoin de nous arrêter lorsque la première cellule visitée est à nouveau visitée.

Pour parcourir la liste à partir de la cellule courante, on doit alors conserver l'adresse de la cellule courante afin de pouvoir la reconnaître et savoir que le parcours est terminé.

Nous n'utilisons pas le pointeur courant pour ce parcours mais un autre pointeur intermédiaire « ll » qui prend la valeur de courant au départ.

L'adresse de la liste ne bouge donc pas.

Version 1 du parcours itératif

Précondition : non estVideLC(l)

```
void itParcours(Liste l) {  
    Liste ll;  
    int arret ;  
    ll=l;  
    arret ← faux  
    while(non arret ) {  
        traiter(donnee(ll));  
        ll =suivant(ll);  
        arret = (ll==l);  
    }  
}
```

Version 2 du parcours itératif**Précondition : non estVideLC(l)**

```

void itParcours(Liste l) {
    Liste ll;
    ll=l;
    traiter(donnee(ll));
    ll=suivant(ll);
    while(ll != l){
        traiter(donnee(ll));
        ll=suivant(ll);
    }
}

```

Version 3 du parcours itératif**Précondition : non estVideLC(l)**

```

void itParcours(Liste l) {
    Liste ll;
    ll=l;
    do{
        traiter(donnee(ll));
        ll=suivant(ll);
    } while(ll != l);
}

```

Version Récursive**Précondition : non estVideLC(l)**

```

void reParcours(Liste ll, Liste l){
    if(ll != l){
        traiter(donnee(ll));
        ll=suivant(ll);
        reParcours(ll, l) ;
    }
}

```

A l'appel :

```

if (!estVideLC(l)) {
    traiter(donnee(l));
    reParcours(suivant(l), l) ;
}
else{
    ...
}

```

Algorithmes d'insertions

Insertion d'un élément dans une liste vide

Fonction allocMemRempCel : TElement → Liste

```

Liste allocMemRempCel(TElement elt){
Liste cel ;

    cel=(ListeC)malloc(sizeof(struct cellule)) ;

    // tester l'allocation

    If (estVideLC(cel))

        rerror(« pb d'alloc mémoire) ;

    cel->donnee=elt ;

    return cel ;

}

```

Insertion d'un élément dans une liste circulaire vide

La Liste pointe sur elle-même :



Fonction inserListeCV : TElement → Liste

```

Liste inserListeCV(TElement elt){
Liste cel;

    // Création de la cellule et remplissage du membre donnee
    cel = allocMemRempCel(elt) ;

    //réalisation du chaînage
    cel->suivant = cel;

    return cel ;

}

```

Insertion avant la cellule courante 'l' d'un élément 'elt' dans une liste chaînée circulaire non vide

Fonction inserAvantLC : TElementxListe → Liste

Précondition : non est VideLC(l)

Principe

- 1) Création d'une cellule et remplissage du membre donnée de la cellule.
- 2) **Détermination de l'adresse d'insertion.**
- 3) Réalisation des chaînages (A) et (B).

Avant l'insertion de l'élément 13

Après insertion de l'élément 13

```
Liste inserAvantLC(TElement elt, Liste l){
Liste cel, ai;
    // Création de la cellule et remplissage du membre donnée
    cel = allocMemRempCel(elt) ;
    //Recherche de l'adresse d'insertion
    ai = adrInsertion(l);
    //Réalisation du chaînage (A)
    cel->suivant = l;
    //Réalisation du chaînage (B)
    ai->suivant = cel;
    return l ;
}
```

Adresse d'insertion = est celle avant la cellule l

Précondition : non est VideLC(l)

```
Liste adrInsertion(Liste l){
Liste ai ;

    //Parcours de la Liste l
    ai = l ;
    while (suivant(ai) != l)
        ai = suivant(ai);
    return ai;
}
```

Insertion après la cellule courante 'l' d'un élément 'elt' dans une liste chaînée circulaire non vide

Fonction inserApresLC : TElementxListe → Liste

Précondition : non est VideLC(l)

Principe

- 1) Création d'une cellule et remplissage du membre donnée de la cellule.
- 2) L'adresse d'insertion est le suivant l'adresse courante.
- 3) Réalisation des chaînages (A) et (B) = L'insertion en tête par rapport au suivant de la cellule courante.

Avant l'insertion de l'élément 13

Après l'insertion de l'élément 13


```
Liste inserApresLC (TElement elt, Liste l){
```

```
Liste cel;
```

```
// Creation de la cellule et remplissage du membre donne
```

```
cel = allocMemRempCel(elt) ;
```

```
// Adresse d'insertion = suivant(l)
```

```
//réalisation du chaînage (A)
```

```
cel->suivant = suivant(l);
```

```
// réalisation du chaînage (B)
```

```
l->suivant = cel;
```

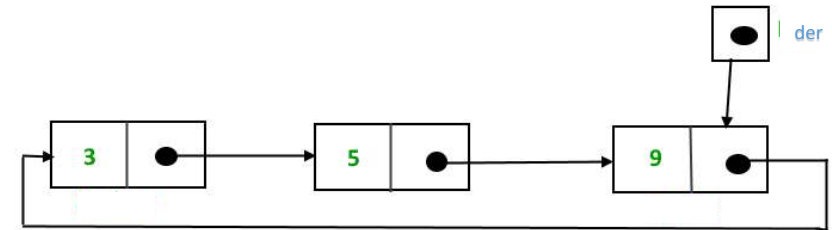
```
return l ;
```

```
}
```

Choix d'implémentation

Pour implémenter une liste simple circulaire, nous prenons l'adresse de la dernière cellule comme adresse de la liste circulaire.

Si **der** est l'adresse de la dernière cellule, alors **suivant(der)** est l'adresse de la première cellule.



Pourquoi l'adresse de la dernière cellule est non l'adresse de la première cellule :

Si nous prenons l'adresse de la première cellule comme adresse de la liste : **l'insertion en tête ou en queue nécessite le parcours total de la liste.**

Si nous prenons l'adresse de la dernière cellule comme adresse de la liste : **l'insertion en tête ou en queue ne nécessite pas le parcours total de la liste.**

C/C : L'insertion en tête ou en queue prend un temps constant quelle que soit la longueur de la liste.

Utile pour l'implémentation d'une file d'attente : premier arrivé est le premier servi.

Insertion en tête

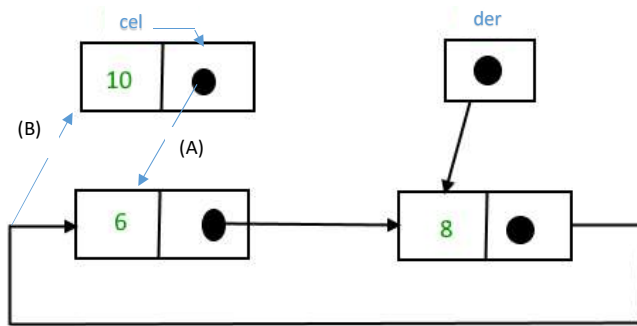
Principe

1) Création d'une cellule et remplissage du membre donnée de la cellule.

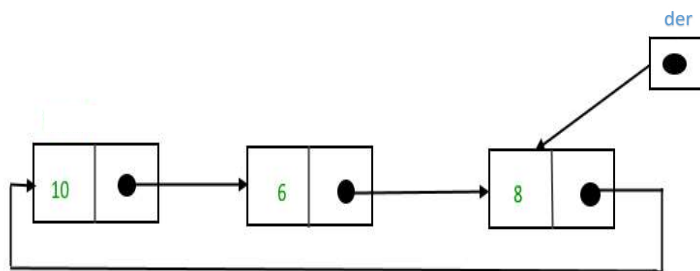
2) L'adresse d'insertion est le suivant de l'adresse courante (cad **suivant(der)**).

3) Réalisation des chaînages (A) et (B)

Avant l'insertion :



Après insertion :



Insertion d'un élément en tête dans une liste circulaire non vide

```
Liste inserTeteLC(TElement elt, Liste der){
    Liste cel;
```

```
    // Création de la cellule et remplissage du membre donne
    cel = allocMemCellule(elt) ;
```

```
    //realisation du chaînage (A)
    cel->suivant = suivant(der);
```

```
    // réalisation du chaînage (B)
    der->suivant = cel;
```

```
    return der ;
```

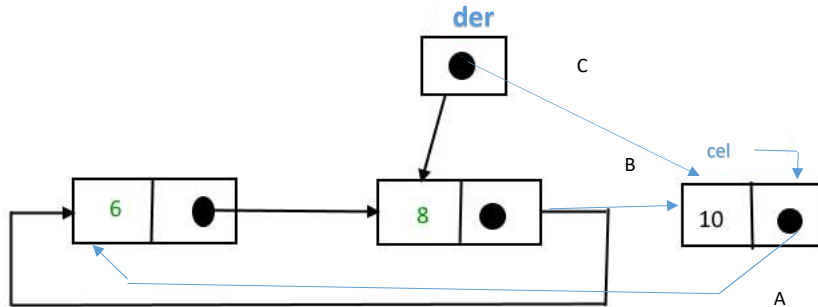
```
}
```

Insertion en queue

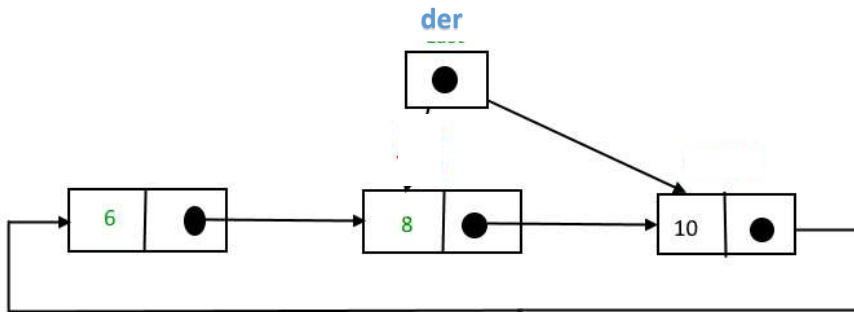
Principe

- 1) Création d'une cellule et remplissage du membre donnée de la cellule.
- 2) L'adresse d'insertion est le suivant de l'adresse courante (cad `suivant(der)`).
- 3) Réalisation des chaînages (A), (B) et (C)

Avant l'insertion de 10 :



Après l'insertion de 10 :



Insertion d'un élément en queue dans une liste circulaire non vide

```
Liste inserQueueLC(TElement elt, Liste der){
Liste cel ;
```

```
//Création de la cellule et remplissage du membre donne
Cel = allocMemCellule(elt) ;
```

```
//Réalisation du chaînage (A)
Cel->suivant = suivant(der);
```

```
//Réalisation du chaînage (B)
der->suivant = cel;
```

```
//Réalisation du chaînage (C)
der = cel ;
return der ;
```

```
}
```

Algorithmes de suppressions

La suppression se fait après la cellule courante et non au début ou à la fin.

Suppression de la cellule qui suit la cellule courante

→ Adresse suppression = suivant(l)

Hypothèse : on suppose que la liste est non vide.

Principe:

- 1) Préserver l'adresse de la cellule à supprimer.
- 2) Modification des chaînages.
- 3) Si la cellule à supprimer est aussi la courante (c'est à dire que la liste est réduite à une seule cellule). Alors la liste devient une liste vide.
- 4) Dans tous les cas libérer la mémoire de la cellule à supprimer.

Avant la suppression

Après la suppression

```
Liste suppSuiv(Liste l) {
    Liste as ;
    // Préserver la cellule à supprimer
    as=suivant(l);
    //Modification chaînages
    if (l==as)
        l=NULL;
    else
        l->suivant=suivant(as);
    // Libération mémoire
    free(as);
    return l ;
}
```

Suppression de la liste

Hypothèse : On suppose que la liste est non vide.

Principe

Pour détruire la liste, il faut parcourir autrement la liste.

On parcourt la liste à partir de la seconde cellule jusqu'à retrouver la première.

Le traitement de la cellule courante est donc l'application de l'algorithme `suppTete` par rapport à cette cellule.

```
void itSuppListe(Liste *l) {
    Liste cel, ll;

    ll=suivant(*l);
    while (ll!=*l){
        cel=ll;           (1)
        ll=suivant(ll);   (2)
        free(cel);        (3)
    }
    //Suppression de la première cellule
    free(*l);
    *l=NULL;
}
```

(1), (2), (3) <-> appel `suppTete(ll)`

```
void itSuppListe(Liste *l) {
    Liste cel, ll;

    ll=suivant(*l);
    while (ll!=*l){
        ll=suppTete(ll);
    }
    //Suppression de la première cellule
    free(*l);
    *l=NULL;
}
```

Version récursive

```
void reSuppListe(Liste *ll, Liste l){  
  
    if (*ll!=1){  
        cel=*ll;           (1)  
        *ll=suivant(*ll);  (2)  
        free(cel);         (3)  
        reSuppListe(ll, l)  
    }  
}
```

(1), (2), (3) <-> appel suppTete(ll)

```
void reSuppListe(Liste *ll, Liste l){  
  
    if (*ll!=1){  
        *ll=suppTete(*ll);  
        reSuppListe(ll, l)  
    }  
}
```

A l'appel :

```
ll = suivant(l) ;  
reSuppListe(&ll, l);
```

```
//Suppression de la première cellule
```

```
free(l);  
l=NULL;
```